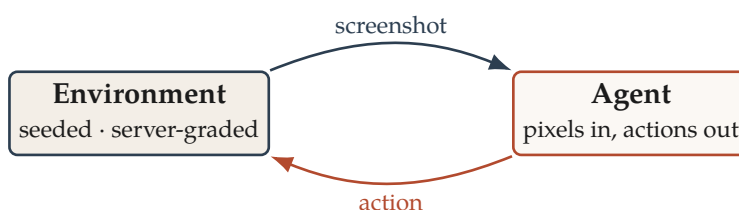

A TEXTBOOK ON THE CONTROLLED EVALUATION OF GUI AGENTS

WebTask Arena

*Where, and why, do agents that operate
software interfaces fail?*



Xining Li

First edition · July 5, 2026

github.com/xiningli/webtask-arena

WebTask Arena: A Textbook on the Controlled Evaluation of GUI Agents. First edition.

This book documents the design, implementation, and use of WebTask Arena, an open-source environment for the reproducible evaluation of computer-use agents. Every code listing is drawn from the accompanying source tree; file paths are given as `server/app.py`-style references so that each idea can be traced to the code that implements it.

The software described here is released under the MIT License. This text may be freely used for study and teaching.

Companion repository: github.com/xiningli/webtask-arena

Live demo (Hugging Face Space): huggingface.co/spaces/xiningli/webtask-arena

UI-TARS evaluation: github.com/xiningli/uitars-webtask-eval

Preface

A new class of AI system has arrived: models that do not merely *describe* how to use software but actually operate it, reading a screen and emitting mouse clicks and keystrokes the way a person would. These *computer-use* or *GUI agents* promise to automate the long tail of digital work that never had an API. But a capability is only as trustworthy as our ability to measure it, and measuring a GUI agent turns out to be genuinely hard. The agent’s world is a living web page: it shifts, it lags, its buttons move, and its notion of “success” is buried in application state the agent never directly sees.

This book is about building an instrument for that measurement. WebTask Arena is a small, fully owned collection of web tasks engineered so that every source of noise an evaluator normally fights — nondeterminism, brittle rewards, external dependencies, ambiguous goals — has been designed out. What remains is a clean signal: given a fixed task and a fixed seed, *does this agent succeed, and if not, which specific capability failed?*

Who this book is for. It is written for the engineer or researcher who wants to evaluate, debug, or train agents that act on interfaces, and who would rather understand an evaluation harness from its foundations than treat it as a black box. We assume comfort with Python and a working mental model of how web pages and HTTP work. No prior background in reinforcement learning is required; where RL concepts appear, they are introduced from scratch, because one of this book’s arguments is that a rigorous *environment* is the unglamorous prerequisite for any serious RL or evaluation effort.

How to read it. Part I motivates the whole enterprise and states the design principles the rest of the book applies relentlessly. Part II builds the environment — the server, its determinism guarantees, the task abstraction, and the reward functions. Part III builds the harness that drives real browsers. Part IV covers the agents, from a reference oracle to a live Claude adapter. Part V is about evidence: the test suite, a real failure case study, and the vocabulary of a failure taxonomy. Part VI looks outward, to reinforcement learning and open research. The appendices are a reference you will return to.

Every chapter opens with objectives and closes with exercises; the boxes throughout mark the ideas worth remembering, the principles worth internalizing, and the pitfalls worth avoiding. Read with the source open in the other window.

— X.L.

Contents

Preface	iii
I Foundations	1
1 The Problem: Why GUI Agents Are Hard to Evaluate	2
1.1 What a GUI agent actually does	2
1.2 Four sources of variance	3
1.3 Why public benchmarks make controlled study hard	3
1.4 The thesis of this book	4
2 Design Principles of a Controlled Arena	5
2.1 Principle 1 — Every episode is seed-deterministic	5
2.2 Principle 2 — Reward is computed server-side from ground truth	5
2.3 Principle 3 — Success decomposes into named subgoals	6
2.4 Principle 4 — Adversarial rendering is a flag, not a fork	6
2.5 Principle 5 — Hermetic by construction	6
2.6 A map of the implementation	7
II The Environment	8
3 Server Architecture and the Episode Lifecycle	9
3.1 The two surfaces	9
3.2 The lifecycle of one episode	10
3.3 Where episodes live	10
3.4 The request model and defaults	11
4 Determinism and Seeding	13
4.1 One generator, keyed by task and seed	13

4.2	Data pools: distinctness by design	14
4.3	Proving it: the determinism contract	14
4.4	The task author’s contract	15
5	The Task Protocol	16
5.1	A protocol, not a base class	16
5.2	The five responsibilities	17
5.3	Registration: one list, many consumers	17
5.4	Anatomy of adding a task	18
6	Three Tasks and the Craft of Reward Design	19
6.1	Email-triage: precision as a first-class citizen	19
6.2	Settings-panel: a draft is not a commitment	20
6.3	Checkout-form: when the UI lies	21
6.4	Patterns of good reward design	22
7	Adversarial Rendering	24
7.1	Perturbation as a parameter	24
7.2	What each perturbation stresses	25
7.3	Running the ablation	25
8	Sessions, Concurrency, and Hermeticity	27
8.1	Why sessions exist	27
8.2	Isolation, tested	28
8.3	Scaling: more containers, not more sharing	28
III	The Harness	30
9	The Agent Interface and Action Vocabulary	31
9.1	The agent protocol	31
9.2	The action vocabulary	32
9.3	Validation before execution	32
10	The Browser Executor	34
10.1	Pixels in, events out	34

10.2	The fixed viewport	35
10.3	Bridging two key vocabularies	35
10.4	The DOM oracle, and its one rule	35
11	The Episode Runner and Metrics	37
11.1	The main loop	37
11.2	Two channels, never crossed	38
11.3	Artifacts: a complete, replayable record	38
11.4	Metrics: from booleans to a run summary	39
11.5	From pass rate to pass ^k	39
IV	Agents	41
12	The Scripted Oracle Agent	42
12.1	Why an oracle agent exists	42
12.2	Plans as generators	42
12.3	The settings plan and a subtle lesson	43
13	The Claude Computer-Use Adapter	45
13.1	Zero-translation by design	45
13.2	Threading screenshots and tool results	45
13.3	The boundary, enforced by construction	46
14	Porting New Agents	48
14.1	The shape of the work	48
14.2	Translation pitfalls	49
V	Evaluation and Findings	50
15	The Contract Test Suite	51
15.1	Determinism and sanity	51
15.2	Testing the traps	52
15.3	Validation and ordering	52
16	Case Study: UI-TARS-1.5-7B in the Arena	54

16.1	The result, in one line each	54
16.2	Anatomy of a model failure	54
16.3	Anatomy of an environment finding	55
16.4	Why one seed still teaches	55
17	Toward a Failure Taxonomy	57
17.1	Four categories	57
17.2	From subgoals to categories	58
17.3	Why a taxonomy beats a score	58
17.4	Building the taxonomy pipeline	58
18	Replays and the Human Baseline	60
18.1	Packaging a trajectory for replay	60
18.2	The same-seed human baseline	60
18.3	The demo as pedagogy	61
VI	Extensions and Research	62
19	From Evaluation to Reinforcement-Learning Environments	63
19.1	The arena is already most of an RL environment	63
19.2	What RL would add, and what it would demand	64
19.3	Determinism in training versus evaluation	64
20	Research Directions	66
20.1	More agents, more coverage	66
20.2	Richer metrics: the seed $\times$ condition grid	66
20.3	Harder interactions	66
20.4	The standing deliverable	67
VII	Appendices	68
A	HTTP API Reference	69
B	Action Reference	70

C	Command Reference	71
D	Glossary	72

PART I

Foundations

The Problem: Why GUI Agents Are Hard to Evaluate

A GUI agent's competence is a distribution, not a number. To estimate it honestly you must control everything that is *not* the agent: the task, the environment's randomness, the reward, and the rendering. This chapter argues that most evaluation noise is an artifact of the harness, not the agent — and that it can be engineered away.

Learning objectives. After this chapter you will be able to: (i) enumerate the distinct sources of variance in a GUI-agent evaluation; (ii) explain why DOM-scraped rewards are unreliable; and (iii) state the trade-off WebTask Arena makes relative to broad benchmarks such as OSWorld and WebArena.

1.1 What a GUI agent actually does

Strip away the marketing and a computer-use agent is a loop. It receives an observation — almost always a screenshot, a grid of pixels at some fixed resolution — together with a natural-language instruction. It emits an action drawn from a small vocabulary that mimics a human at a keyboard and mouse: `left_click` at a pixel coordinate, `type` some text, `scroll`, press a `key`. The environment applies the action, the screen changes, and the loop repeats until the agent declares itself finished or a step budget is exhausted. Figure 1.1 shows the shape of it.

Two properties of this loop make evaluation difficult, and they are worth naming precisely because the rest of the book is a systematic response to them.

First, the observation is *lossy and unstable*. The same underlying state can render as many different pixel grids: fonts load late, a spinner is mid-spin, an animation is halfway done, a promotional banner slides in and pushes every button down by forty pixels. An agent that clicked correctly a moment ago can miss by clicking the identical coordinate after a layout shift.

Second, success is *not visible in the observation*. Whether an order was placed correctly depends on what the server recorded, not on what the confirmation page says. A page can cheerfully display “Order complete!” for an order shipped to the wrong address. Any grader that reads success off the screen can be fooled by exactly the failure it is supposed to catch.

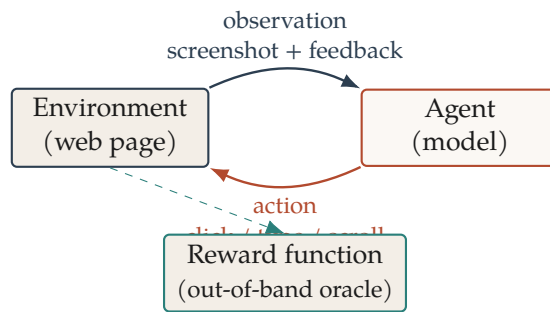


Figure 1.1: The computer-use loop. The agent never sees the reward function; it is computed by an oracle from privileged state after the episode. Keeping the reward *out-of-band* is a recurring theme of this book.

1.2 Four sources of variance

When you run the same agent on the “same” task twice and get different results, the difference comes from one of four places. Only the last is the thing you actually want to measure.

1. **Task variance.** The task itself differs run to run — a different inbox, a different address to type. Desirable for coverage, fatal for debugging if you cannot *reproduce* a specific instance.
2. **Environment variance.** The page renders or behaves differently: network timing, animation state, server version. Pure noise.
3. **Grading variance.** The reward function returns different verdicts for the same final state — a flaky CSS selector, a race against a late-loading element. Pure noise, and the most insidious because it corrupts the label.
4. **Policy variance.** The agent, being a stochastic model, genuinely chose differently. *This* is the signal.

DESIGN PRINCIPLE.

Every source of variance except policy variance is a property of the harness, and therefore something the harness author can eliminate. Reproducibility is not a nice-to-have; it is the precondition for attributing a failure to the agent at all.

1.3 Why public benchmarks make controlled study hard

Benchmarks like OSWorld and WebArena are indispensable: they measure agents against realistic software with broad task coverage, and progress on them is real progress. But their realism is purchased at the cost of control. Tasks depend on specific versions of real applications that drift over time; rewards are often scraped from live UIs and are correspondingly brittle; and a single run can be slow and expensive enough that gathering the repetitions needed for a

confidence interval is painful. When an agent fails, it is frequently unclear whether it failed because the model is weak, because the app updated, or because the grader misfired.

WebTask Arena takes the opposite trade-off, deliberately. It offers a *small* number of tasks that are *completely owned*: the page, the state, the reward, and the randomness all live in one hermetic container with no external dependency. Coverage is sacrificed; control is bought. The two kinds of evaluation are complementary — you use a broad benchmark to ask “how capable, overall?” and an arena like this one to ask “*where*, exactly, and *why*?”

This is not a benchmark-beating project and should never be framed as one. Its output is not a leaderboard number but a *failure taxonomy*: a structured account of the distinct ways agents break. Chapter 17 makes that claim concrete.

1.4 The thesis of this book

Everything that follows is an elaboration of a single sentence: *if you control the task with a seed, compute the reward from privileged server state, decompose success into named subgoals, and make adversarial rendering a toggle rather than an accident, then a failed episode becomes a diagnosis rather than a mystery*. The next chapter turns that sentence into five design principles; the rest of the book implements them.

Exercises

- 1.1 For each of the four sources of variance, give a concrete example drawn from an e-commerce checkout flow you have used. Which two are hardest to eliminate in a benchmark built on a live third-party site, and why?
- 1.2 A colleague proposes grading a “send the email” task by checking that the string “Message sent” appears on screen. Describe two distinct final states that pass this check but represent task failure.
- 1.3 Argue both sides: is sacrificing task coverage for control a reasonable trade-off for a team whose goal is to *improve* an agent (as opposed to *rank* several agents)? What changes if the goal is the latter?

Design Principles of a Controlled Arena

Five commitments define WebTask Arena: seed-determinism, server-side reward, subgoal decomposition, adversarial rendering as a flag, and hermeticity. Hold these five in mind and every implementation decision in the book follows almost mechanically.

Learning objectives. You will be able to state each of the five principles, justify it against the variance analysis of Chapter 1, and recognize its fingerprints in later code.

2.1 Principle 1 — Every episode is seed-deterministic

A *seed* is an integer that fully determines an episode: its data, its instruction, its initial page. Resetting a task with the same seed on any machine must produce byte-identical state.

```
POST /api/email-triage/reset {"seed": 42}
-> identical instruction, identical inbox, every time, everywhere
```

Determinism collapses *task variance* to zero on demand: you can hold the task fixed and vary only the agent, which is exactly what attributing a failure to the policy requires. It also makes bugs shareable — a failing seed is a complete, portable reproduction. Chapter 4 shows the one discipline that buys this: every byte of randomness flows through a single seeded generator keyed by `(task_id, seed)`.

2.2 Principle 2 — Reward is computed server-side from ground truth

The reward function reads the environment's own authoritative state — the Python objects the server mutates as the agent acts — never the rendered DOM.

DESIGN PRINCIPLE.

The agent under evaluation must have no causal path to the grader. If success is read from the same surface the agent manipulates, a sufficiently clever (or sufficiently confused) agent can produce the appearance of success without its substance. Server-side grading closes that path by construction.

This single decision eliminates *grading variance* (no selector can be flaky if no selector is consulted) and defends against reward hacking. It is why, in Chapter 6, the checkout task can distinguish “the UI said the order was placed” from “the order was placed correctly.”

2.3 Principle 3 — Success decomposes into named subgoals

A task does not return a single boolean. It returns a dictionary of named booleans, and succeeds only if all are true.

```
{ "success": false,
  "subgoals": { "archived_all_from_target": true,
                "no_extra_archived": false,
                "starred_target_only": true }}
```

The overall verdict here is *failure*, but the shape of the failure is now legible: the agent archived everything it should have, and starred correctly, but also archived something it should not have. That is a precision error, not a recall error, and the two implicate different capabilities. A scalar reward throws this information away; named subgoals preserve it. Subgoals are the atoms from which the failure taxonomy of Chapter 17 is built.

2.4 Principle 4 — Adversarial rendering is a flag, not a fork

Real interfaces are not instant and stable. To measure how much of an agent’s success is an artifact of ideal conditions, WebTask Arena can inject two perturbations into *any* task without changing its logic:

- `latency_ms` — an artificial delay before the page becomes interactive, simulating load time;
- `layout_shift` — a banner that slides in after the page settles, displacing every element below it.

Because these are request parameters rather than separate task variants, the clean and adversarial conditions share identical task logic and identical reward. Any difference in pass rate is therefore attributable to the perturbation alone — a clean ablation. Chapter 7 implements both in a dozen lines.

2.5 Principle 5 — Hermetic by construction

The container is the whole world. No CDN, no web font, no analytics beacon, no external request of any kind. Hermeticity is what makes determinism *true* rather than aspirational: a page that fetches a third-party script is at the mercy of that third party’s uptime, version, and latency, and no seed can control it. It also makes the environment cheap to parallelize — you scale evaluation by running more identical containers, with nothing shared to contend over.

The five principles reinforce one another. Hermeticity (5) makes determinism (1) achievable; determinism makes the adversarial ablation (4) clean; server-side reward (2) makes subgoals (3) trustworthy. They are not a menu; they are a system.

2.6 A map of the implementation

Table 2.1 previews where each principle lives in the source. Keep it handy; the chapters that follow fill in each row.

Principle	Primary file	Mechanism
Seed-determinism	<code>server/seeding.py</code>	one <code>Random</code> per <code>(task, seed)</code>
Server-side reward	<code>server/tasks/*.py</code>	<code>verify()</code> reads state objects
Named subgoals	<code>server/tasks/*.py</code>	<code>verify()</code> returns a dict
Adversarial rendering	<code>templates/base.html</code>	<code>latency_ms</code> , <code>layout_shift</code>
Hermeticity	<code>Dockerfile</code>	no external hosts, self-contained

Table 2.1: The five principles and where to find them.

Exercises

- 2.1 Principle 2 forbids the grader from reading the DOM. Yet the *agent* sees only the rendered page. Explain why this asymmetry is essential rather than unfair, referring to what each party is being tested for.
- 2.2 Suppose you dropped Principle 5 and allowed one external font CDN. Construct a scenario in which two runs of the same seed diverge. Which of the other four principles is thereby weakened?
- 2.3 Design a sixth principle you believe belongs in this list. State it in one sentence, justify it against a source of variance or a threat, and identify which existing principle it most depends on.

PART II

The Environment

Server Architecture and the Episode Lifecycle

The environment is a small FastAPI application. It exposes two disjoint surfaces: a *task surface* the agent may touch through a browser, and an *oracle surface* (`reset`, `verify`, `state`) that only the harness may touch. Episodes are dictionaries in a session-keyed, LRU-capped store.

Learning objectives. You will be able to trace a full episode through the server's routes, explain the agent/oracle boundary and why it is a security property, and describe how episode state is stored and evicted.

3.1 The two surfaces

The single most important architectural fact about the server is that its routes divide cleanly into two groups with different audiences.

Route	Audience	Purpose
GET <code>/task/{id}</code>	agent (via browser)	the rendered task page
POST <code>/api/{id}/action</code>	agent (via page JS)	apply a UI action
POST <code>/api/{id}/reset</code>	harness only	start a seeded episode
GET <code>/api/{id}/verify</code>	harness only	grade the episode
GET <code>/api/{id}/state</code>	harness only	inspect privileged state

The docstring at the top of `server/app.py` states the rule the whole design depends on:

```
"""
`/api/*/verify` and `/api/*/state` are oracle endpoints for the harness and
must never be exposed to the agent under evaluation.
"""
```

DESIGN PRINCIPLE

Treat the oracle endpoints as a capability the agent must never hold. The harness calls `verify` out-of-band, over a direct HTTP client the model has no access to; the model reaches the server only through the browser, which can touch the task surface and nothing else. This is the runtime enforcement of Principle 2.

3.2 The lifecycle of one episode

An episode moves through four phases. The comment block that opens the app captures them exactly:

```
# POST /api/{task_id}/reset {"seed": 42, "latency_ms": 0, "layout_shift": false}
#   -> {"task_id", "seed", "instruction"}
# ...agent interacts with GET /task/{task_id} through the browser...
# GET  /api/{task_id}/verify
#   -> {"success": bool, "subgoals": {name: bool}}
```

Reset builds fresh state from the seed and stores it. **Interact** is the agent's loop against the task page, each UI action routed through `POST /api/{id}/action` to a task-specific handler that mutates state. **Verify** grades the stored state. A fourth route, **state**, exposes the raw episode for debugging and for the reference agent's self-tests. Here is `reset` and `verify` in full:

```
@app.post("/api/{task_id}/reset")
def reset(task_id: str, req: ResetRequest, request: Request) -> dict[str, Any]:
    if task_id not in TASKS:
        raise HTTPException(404, f"unknown task '{task_id}'")
    sid = _sid(request)
    ep = _do_reset(sid, task_id, req)
    return {"task_id": task_id, "seed": ep["seed"], "instruction": ep["instruction"]}

@app.get("/api/{task_id}/verify")
def verify(task_id: str, request: Request) -> dict[str, Any]:
    ep = _episode(_sid(request), task_id)
    subgoals = TASKS[task_id].verify(ep["state"])
    return {"success": all(subgoals.values()), "subgoals": subgoals}
```

Notice how thin these handlers are. All task-specific logic lives behind the `TASKS[task_id]` object (Chapter 5); the app is a dispatcher. Notice too the definition of success: `all(subgoals.values())`. Success is not a primitive the task returns — it is *derived* from the conjunction of subgoals, which guarantees that “succeeded” and “every subgoal passed” can never disagree.

3.3 Where episodes live

Episode state is held in an ordered dictionary keyed by a `(session, task)` pair, with a hard cap:

```

# (sid, task_id) -> {"state": ..., "seed": ..., "opts": ..., "instruction": ...}
EPISODES: OrderedDict[tuple[str, str], dict[str, Any]] = OrderedDict()
MAX_EPISODES = 512 # LRU cap for public deployments

def _do_reset(sid: str, task_id: str, req: ResetRequest) -> dict[str, Any]:
    task = TASKS[task_id]
    state = task.generate(req.seed)
    EPISODES[(sid, task_id)] = {
        "state": state,
        "seed": req.seed,
        "opts": {"latency_ms": req.latency_ms, "layout_shift": req.layout_shift},
        "instruction": task.instruction(state),
    }
    EPISODES.move_to_end((sid, task_id))
    while len(EPISODES) > MAX_EPISODES:
        EPISODES.popitem(last=False)
    return EPISODES[(sid, task_id)]

```

The `OrderedDict` plus `move_to_end` plus `popitem(last=False)` is a textbook least-recently-used cache: touching an episode marks it fresh, and when the store overflows the oldest untouched episode is evicted. For a harness that runs one episode per container this cap never binds; it exists so that the same code can back a public demo where many visitors mint sessions (Chapter 8).

State is in-memory and per-process. This is a feature: it makes an episode a plain Python object with no serialization boundary between the action handler and the grader, which is what lets `verify` read ground truth directly. The cost — state is lost on restart, and one process is one shard — is exactly the cost a hermetic, container-per-shard design is happy to pay.

3.4 The request model and defaults

Reset accepts a typed body with conservative defaults, so the simplest possible call — no body at all — yields a clean, seed-0, non-adversarial episode:

```

class ResetRequest(BaseModel):
    seed: int = 0
    latency_ms: int = 0
    layout_shift: bool = False

```

Defaulting the adversarial flags to off means the *easy* condition is the one you get for free, and adversity is something you opt into explicitly — which keeps experiments honest about what they measured.

Exercises

- 3.1 The `verify` route computes `all(subgoals.values())` rather than letting each task report its own `success`. Give two distinct bugs this design makes impossible.
- 3.2 Trace, route by route, what happens when the harness runs one `email-triage` episode at seed 7 with `latency_ms=500`. Name every handler entered and every piece of state written.
- 3.3 The LRU cap is 512. A public deployment gets a burst of 600 distinct visitors within one minute, each starting one episode. Describe precisely what happens to visitor #1's episode, and whether any visitor sees another's data.

Determinism and Seeding

Determinism is bought with one rule, enforced by one function: all episode randomness flows through `rng(task_id, seed)`, a `random.Random` seeded by the string `"task::seed"`. Same key, same stream, same episode — on every machine, forever.

Learning objectives. You will understand why a string-seeded PRNG per episode yields cross-machine reproducibility, how the data pools are structured, and what disciplines a task author must follow to keep determinism intact.

4.1 One generator, keyed by task and seed

The entire determinism guarantee rests on eleven lines in `server/seeding.py`:

```
"""Deterministic data pools. All episode randomness must flow through
`rng(task_id, seed)` so identical (task, seed) pairs replay identically."""

import random

def rng(task_id: str, seed: int) -> random.Random:
    return random.Random(f"{task_id}::{seed}")
```

Two design choices are doing quiet but important work here.

First, the generator is *local*, not *global*. `random.Random(...)` constructs a fresh instance with its own state; it never touches the process-wide `random` module. Two tasks generating episodes concurrently cannot perturb each other's streams, and nothing elsewhere in the process can reach in and advance the sequence. Global `random.seed()` would make determinism a property of *global call order*, which is impossible to preserve under concurrency.

Second, the seed is a *string* composed of the task id and the integer seed. Python's `Random` accepts a string and hashes it stably to seed the Mersenne Twister. Keying on `task_id` as well as the number means seed 0 of `email-triage` and seed 0 of `checkout-form` are independent streams — a coincidence of layout between two tasks at the “same” seed is impossible by construction.

DESIGN PRINCIPLE

Determinism is not a property you test for; it is a property you *architect* by routing every nondeterministic choice through a single seeded source. If any code path calls the global `random`, reads the clock, or hashes an address, determinism is already lost — no amount of testing will reliably catch it.

4.2 Data pools: distinctness by design

The randomness draws from curated pools — names, email domains, subject topics, timezones, products. The pools are not arbitrary; several are shaped to keep generated instructions *unambiguous*. Consider how the email task draws subjects:

```
topics = r.sample(TOPICS, N_EMAILS) # distinct topics => unambiguous keyword
```

`r.sample` draws *without replacement*, so all twelve emails in an inbox have distinct subjects. This matters because the instruction later says “star the email with the subject ‘Holiday schedule’”; if two emails shared that subject the instruction would be ambiguous and the reward ill-defined. The determinism discipline and the reward’s well-definedness are thus entangled: the seeding module is quietly protecting the grader.

A helper such as `person(r)` draws a first name, last name, and domain from the pools and composes an email address. Because it takes the generator `r` as an argument rather than creating its own, it participates in the single episode stream — the same discipline, one level down. Every helper that consumes randomness must take the `Random` as a parameter; none may manufacture its own.

4.3 Proving it: the determinism contract

Determinism is a claim, and claims about behavior deserve tests. The suite in `tests/test_environment.py` asserts it directly, parametrized across tasks and seeds:

```
@pytest.mark.parametrize("task_id", ALL_TASKS)
@pytest.mark.parametrize("seed", [0, 7, 12345])
def test_same_seed_same_episode(task_id, seed):
    first = reset(task_id, seed)
    first_state = oracle(task_id)
    second = reset(task_id, seed)
    assert first["instruction"] == second["instruction"]
    assert first_state == oracle(task_id)
```

Resetting twice with the same seed must reproduce both the natural-language instruction and

the full privileged state. A companion test, `test_different_seeds_differ`, asserts the complementary property — that twenty different seeds produce more than one distinct instruction — so that a determinism bug that froze every seed to the same episode could not hide.

4.4 The task author's contract

Determinism survives only if every task obeys the discipline. In practice this reduces to three rules for anyone adding a task:

1. Obtain your generator exactly once, via `rng(self.id, seed)` inside `generate`.
2. Make every random choice — data, ordering, target selection — a call on that generator. Never call the global `random`, never read the clock, never depend on dict iteration order that is not itself seeded.
3. Let `generate` return a fully materialized state. Do not defer random choices to `view` or `handle_action`; by the time `generate` returns, the episode must be completely determined.

Rule 3 is subtle and worth dwelling on. If a task deferred, say, the choice of which email to star until the page was first rendered, then determinism would depend on *when* the page was first rendered — and two harnesses that screenshot at different moments could disagree. Front-loading all randomness into `generate` makes the episode a value, fixed at reset, that later phases only read and mutate deterministically in response to agent actions.

Exercises

- 4.1 Explain why `random.Random(f"{task_id}::{seed}")` makes seed 0 of two different tasks independent, whereas a global `random.seed(seed)` at the start of each `generate` would not, under concurrent generation.
- 4.2 You are adding a task that shuffles a list of options. Show the one-line correct implementation and the tempting one-line wrong implementation, and state what breaks in the wrong one.
- 4.3 `test_same_seed_same_episode` compares full state, not just the instruction. Construct a determinism bug that the instruction comparison alone would miss but the state comparison catches.

The Task Protocol

A task is any object satisfying a five-method Protocol: `generate`, `instruction`, `view`, `handle_action`, `verify`. Register it in one list and the server, the tests, and the harness all pick it up. The protocol *is* the extension point of the whole system.

Learning objectives. You will be able to read the Task protocol as a contract, explain the responsibility of each method, and understand how registration wires a task into every consumer at once.

5.1 A protocol, not a base class

WebTask Arena defines what a task *is* structurally, using `typing.Protocol` rather than inheritance:

```
class Task(Protocol):
    id: str
    title: str
    template: str

    def generate(self, seed: int) -> dict[str, Any]:
        """Build a fresh, fully seed-determined episode state."""
    def instruction(self, state: dict[str, Any]) -> str:
        """Natural-language goal handed to the agent out-of-band."""
    def view(self, state: dict[str, Any]) -> dict[str, Any]:
        """Template context for rendering the task page."""
    def handle_action(self, state, payload) -> dict[str, Any]:
        """Mutate state in response to a UI action. Return {'ok': bool, ...}."""
    def verify(self, state: dict[str, Any]) -> dict[str, bool]:
        """Named subgoals; episode succeeds iff all are true."""
```

Using a protocol rather than an abstract base class means a task class need only *have the right shape* — three attributes and five methods — to qualify. No import of a framework base, no inheritance, no registration decorator. The three concrete tasks are plain classes that happen to match; a type checker verifies the match statically.

DESIGN PRINCIPLE

Define your extension point by the narrowest interface that the consumers actually require, and prefer structural typing so that implementers owe you conformance, not lineage. A task author's entire obligation is visible in one screen of protocol; nothing is hidden in a superclass.

5.2 The five responsibilities

Each method owns exactly one phase of the lifecycle, and the boundaries are strict.

generate(seed) is the *only* place randomness enters. It returns the complete privileged state as a dictionary — the source of truth that every other method reads. This is where the determinism discipline of Chapter 4 is exercised.

instruction(state) renders the goal into the natural language the agent will receive. It is a pure function of state, so the same episode always yields the same words.

view(state) projects state into the subset the *page* may show. This projection is a security boundary in miniature: the email task's **view**, for instance, exposes only un-archived emails and never the identity of the star target, so the answer is not sitting in the page source.

handle_action(state, payload) is the transition function. It takes the current state and a UI action, validates it, mutates state, and returns an *ok/error* result. It is the one method that *writes* state after reset, and it is where a task encodes what the interface will and will not let a user do.

verify(state) is the reward function: a pure read of state into a dictionary of named subgoals. It never mutates, never renders, never reads the DOM — it inspects the same authoritative objects **handle_action** wrote.

Notice the clean split between what the agent may see (**view**) and what the grader may see (**verify**). Both are pure functions of the same state, but they read different projections of it. **view** is deliberately lossy; **verify** is deliberately complete. Keeping them as separate methods over shared state is what makes “the agent cannot see the answer, but the grader can see everything” true at the level of code.

5.3 Registration: one list, many consumers

Wiring a task into the system is a single edit:

```
from .tasks.checkout_form import CheckoutForm
from .tasks.email_triage import EmailTriage
from .tasks.settings_panel import SettingsPanel

_ALL = [EmailTriage(), SettingsPanel(), CheckoutForm()]
```

```
TASKS: dict[str, Task] = {t.id: t for t in _ALL}
```

That `TASKS` dictionary is the seam the entire rest of the system hangs on. The server dispatches every route through it. The contract tests (Chapter 15) are parametrized over `ALL_TASKS` and so a new entry is automatically subjected to every determinism and reward test. The harness’s scripted agent looks up a plan by the same id. Add a class to `_ALL` and it is, in one motion, served, graded, tested, and drivable.

5.4 Anatomy of adding a task

Putting the pieces together, the full recipe for a new task is:

1. Write a class with `id`, `title`, `template` and the five methods, obeying the seeding discipline in `generate` and the no-DOM rule in `verify`.
2. Add an HTML template that renders `view`’s output and posts actions to `/api/{id}/action`.
3. Append an instance to `_ALL` in `server/registry.py`.
4. Run `pytest`; the parametrized contract tests now cover it.

No other file must change. That property — that the cost of a new task is one class, one template, and one list entry — is the practical payoff of defining the extension point as a narrow protocol.

Exercises

- 5.1 `view` and `verify` are both pure functions of `state`, yet the book insists they must stay separate. Write the two-sentence argument for why merging them (a single method returning everything) would violate Principle 2.
- 5.2 Sketch the state dictionary, instruction string, and three subgoal names for a new “compose and send an email to a named recipient with a given subject” task. Which existing task is it most structurally similar to?
- 5.3 The protocol has no `reset` method, yet episodes reset. Where does reset happen, and why is it correct that the *task* does not implement it?

Three Tasks and the Craft of Reward Design

The three v0 tasks are not toys; each is engineered around a specific failure mode. Email-triage tests precision (extra actions fail). Settings-panel tests commitment (a draft is not a save) and non-interference. Checkout-form tests correctness under a UI that rewards mere completion. Their reward functions are the most instructive code in the repository.

Learning objectives. You will be able to read each task’s `verify` as a precise specification of success, articulate the trap each task sets, and generalize the reward-design patterns they embody.

6.1 Email-triage: precision as a first-class citizen

The instruction is: *archive every email from a target sender, and star one specific email, and do nothing else*. The reward makes “nothing else” enforceable through set algebra:

```
def verify(self, state):
    target = state["target_sender"]
    archived = {e["id"] for e in state["emails"] if e["archived"]}
    should_archive = {e["id"] for e in state["emails"] if e["sender"] == target}
    starred = {e["id"] for e in state["emails"] if e["starred"]}
    return {
        "archived_all_from_target": should_archive <= archived,    # recall
        "no_extra_archived":      archived <= should_archive,    # precision
        "starred_target_only":    starred == {state["star_target_id"]},
    }
```

The two archive subgoals are the two halves of a set equality split apart on purpose. `should_archive <= archived` is *recall*: did you archive everything you should have? `archived <= should_archive` is *precision*: did you archive *only* what you should have? Their conjunction is exactly `archived == should_archive`, but keeping them separate turns a failure into a diagnosis: a missed email fails recall, a spurious archive fails precision, and the two are different bugs implicating different capabilities (Chapter 17 calls the latter a common consequence of poor state tracking).

An agent that archives the whole inbox achieves perfect recall and scores `archived_all_from_target = true`. It is precisely `no_extra_archived` that catches it. Any reward that collapsed the two into one boolean, or that only checked “were the target emails archived?”, would score this catastrophe as a partial success. Precision must be graded explicitly or it is not graded at all.

The task’s construction also protects the reward. Exactly three of the twelve emails come from the target sender; the star target is chosen from emails *not* from that sender, so “archive these” and “star this” can never collide on the same email. And because `view` returns only un-archived emails, the page reflows as emails are archived — the very dynamic that, in Chapter 16, breaks an agent that clicks fixed coordinates.

6.2 Settings-panel: a draft is not a commitment

The instruction is: *enable Dark Mode, set the timezone to a target value, save, and change nothing else*. The reward:

```
def verify(self, state):
    saved = state["saved"]
    untouched = [k for k in DEFAULTS if k not in ("dark_mode", "timezone")]
    return {
        "dark_mode_saved": saved["dark_mode"] is True,
        "timezone_saved": saved["timezone"] == state["target_timezone"],
        "no_unrelated_changes": all(saved[k] == DEFAULTS[k] for k in untouched),
    }
```

The critical word is `saved`. The state distinguishes a *draft* — what the UI currently shows — from what has been committed, and the reward reads only the committed values. The action handler makes the commitment explicit: it accepts only a `save` operation, applies the whole draft at once, and rejects any field it does not recognize:

```
def handle_action(self, state, payload):
    if payload.get("op") != "save":
        return {"ok": False, "error": "unknown op"}
    draft = payload.get("draft", {})
    unknown = set(draft) - set(DEFAULTS)
    if unknown:
        return {"ok": False, "error": f"unknown fields: {sorted(unknown)}"}
    state["saved"].update(draft)
    return {"ok": True}
```

Two traps live here. The first is the draft/commit gap: an agent that toggles Dark Mode in the UI and then declares victory without clicking Save has changed nothing the grader can see — `dark_mode_saved` stays false. This mirrors a real and common human error, and it tests

whether the agent understands that some interface changes require an explicit commit. The second trap is interference: the settings page shows eight toggles across four sections, and `no_unrelated_changes` fails if *any* of the six the agent was told to leave alone drifts from its default. Doing the task is not enough; doing *only* the task is the task.

The timezone control is a native HTML `<select>`. This innocuous choice produces one of the book's most interesting empirical findings: under headless Chromium, the operating system renders the dropdown's option list *outside* the page canvas, so a screenshot-only agent literally cannot see the options it must choose among. That is not an agent bug; it is an *environment finding*, and Chapter 16 treats it as a first-class result.

6.3 Checkout-form: when the UI lies

The instruction is: *complete a three-step checkout — contact, shipping, review — for a specific buyer and address, then submit*. The reward compares the submitted data, field by field, against the expected values fixed at generation:

```
def verify(self, state):
    exp = state["expected"]
    contact, shipping = state["contact"] or {}, state["shipping"] or {}
    return {
        "contact_correct": (_norm(contact.get("name", "")) == _norm(exp["name"]))
            and _norm(contact.get("email", "")) == _norm(exp["email"])),
        "shipping_correct": (_norm(shipping.get("street", "")) == _norm(exp["street"]))
            and _norm(shipping.get("city", "")) == _norm(exp["city"])
            and shipping.get("zip", "").strip() == exp["zip"],
        "submitted": state["submitted"] is True,
    }
```

The trap this task is built around is stated in its own test's name: *wrong data submits but fails verify*. The UI happily accepts a syntactically valid but *incorrect* order — a real name, a real-looking email, a five-digit ZIP that simply is not the right one — and shows a success state. Only the server, comparing against the expected values it generated, knows the order is wrong. `submitted` can be true while `success` is false. This is Principle 2 in its purest form: reward read from the UI would grade this as a win.

The task also encodes genuine multi-step structure. `handle_action` enforces ordering — you cannot submit shipping before contact is accepted, and cannot submit the order before both prior steps are complete — and validates each step server-side, returning field-level errors for a missing name, a malformed email, or a ZIP that is not exactly five digits. Recovering from those validation errors is itself part of what the task measures.

```
if op == "shipping":
    if state["contact"] is None:
```

```

    return {"ok": False, "errors": {"_": "Complete the contact step first."}}
    ...
    if not ZIP_RE.match(str(payload.get("zip", "")).strip()):
        errors["zip"] = "ZIP code must be exactly 5 digits."

```

The `_norm` helper — lowercase and collapse whitespace — is a small but important reward-design decision. It makes grading robust to cosmetically irrelevant differences (a double space, a stray capital) while the ZIP, compared exactly, stays strict where strictness matters. Choosing which fields tolerate normalization and which do not *is* reward design.

6.4 Patterns of good reward design

Across the three tasks, several transferable patterns recur.

DESIGN PRINCIPLE.

Grade both what was done and what was not. Every task pairs an achievement subgoal with a restraint subgoal: archive-all with no-extra-archived, do-the-two-settings with change-nothing-else, submit-correctly with the-data-must-match. Restraint is where naive agents fail and naive rewards are silent.

DESIGN PRINCIPLE.

Separate the axes of failure. Split a compound success into the smallest independently meaningful booleans — recall from precision, commit from content, completion from correctness — so the reward vector localizes the fault.

DESIGN PRINCIPLE.

Normalize deliberately, not reflexively. Tolerate differences that are not part of the task (whitespace, case) and stay exact on the parts that are (the ZIP, the target timezone, the star target). The pattern of normalization encodes what the task considers “the same answer.”

Exercises

- 6.1 The email reward could be written as the single subgoal `archived == should_archive`. Rewrite it that way, then explain in terms of the failure taxonomy what diagnostic information the rewrite destroys.
- 6.2 Settings-panel grades `no_unrelated_changes` over six fields. Suppose a future task has fifty unrelated fields. Does the pattern still scale? What, if anything, would you change about how “unrelated” is expressed?
- 6.3 Design a fourth subgoal for checkout-form that would catch an agent which submits the

correct data but does so by skipping the review step's confirmation. What state would you need `handle_action` to record to make it gradable server-side?

- 6.4** For each task, name the single line in `verify` that most directly embodies Principle 2 (reward from ground truth, not the DOM), and justify your choice.

Adversarial Rendering

Two flags — `latency_ms` and `layout_shift` — perturb rendering without touching task logic or reward. Because they are request parameters shared by the clean and adversarial conditions, any pass-rate gap between conditions is a clean measurement of the perturbation's effect. Both are implemented in the base template's boot script.

Learning objectives. You will understand how the two perturbations are injected, why implementing them client-side keeps the ablation clean, and what agent capability each one stresses.

7.1 Perturbation as a parameter

Recall Principle 4: adversarial rendering is a flag, not a fork. The reset request carries `latency_ms` and `layout_shift`; the server stores them in the episode's `opts` and hands them to the page as a JSON blob. The base template's boot script reads them and stages the page accordingly:

```
const OPTS = ({ opts_json | safe });
window.addEventListener('DOMContentLoaded', () => {
  setTimeout(() => {
    document.getElementById('loading').remove();
    document.getElementById('app').classList.remove('hidden');
    if (OPTS.layout_shift) {
      setTimeout(() => {
        const b = document.createElement('div');
        b.className = 'banner';
        b.textContent = 'New features are available - see what changed...';
        document.body.prepend(b);
      }, 1200);
    }
  }, OPTS.latency_ms || 0);
});
```

Every task page begins life hidden behind a spinner. After `latency_ms` milliseconds the spinner is removed and the app revealed — this is the artificial load latency. If `layout_shift` is set, a further 1.2 seconds later a banner is prepended to the body, shoving every element below it

downward.

DESIGN PRINCIPLE.

Implement perturbations at the presentation layer, downstream of everything the reward depends on. Because `latency_ms` and `layout_shift` change only *when* and *where* pixels appear — never the state or the grading — the clean and adversarial runs are the same experiment with one variable moved. That is what licenses attributing a pass-rate change to the perturbation.

7.2 What each perturbation stresses

Latency stresses the agent’s model of readiness. An agent that acts before the page is interactive clicks into a spinner and accomplishes nothing; the capability under test is patience — waiting for, and recognizing, a settled page. The harness cooperates honestly: when it opens a task it waits for the `#app` element to become visible rather than assuming instant load (Chapter 10), so latency tests the *agent’s* readiness model, not the harness’s.

Layout shift stresses the agent’s habit of grounding actions in the *current* screen. An agent that plans a sequence of clicks against an early screenshot, then executes them after the banner has displaced everything, will click the wrong targets. The capability under test is re-grounding — taking a fresh observation and relocating targets rather than trusting stale coordinates. This is the exact failure the case study in Chapter 16 observes, compounded there by the list reflow that archiving already causes.

The scripted reference agent (Chapter 12) passes all three tasks under both clean and adversarial rendering. It does so not because it is robust in any interesting cognitive sense, but because it re-locates targets through the DOM on every action and the harness waits out latency. That the *oracle* survives adversity confirms the perturbations attack perception and grounding, not task solvability — the tasks remain solvable; seeing them is what gets harder.

7.3 Running the ablation

Because the flags are harness arguments, the ablation is two commands whose only difference is the condition:

```
python -m harness.run --agent claude --task all --seeds 0-4
python -m harness.run --agent claude --task all --seeds 0-4 \
    --latency-ms 800 --layout-shift
```

Subtract the pass rates and you have measured how much of the agent’s success was an artifact of instant, stable pages — a number no single-condition benchmark can report.

Exercises

- 7.1 Why is it important that `layout_shift` fires *after* the page is revealed rather than being present from the first paint? What agent behavior would a from-first-paint banner fail to stress?
- 7.2 The banner is injected client-side by the page's own script. Argue that this cannot affect `verify`'s output for any of the three tasks, referring to what `verify` reads.
- 7.3 Propose a third perturbation flag in the spirit of the existing two. Specify where it would be implemented, what capability it stresses, and why it leaves the reward untouched.

Sessions, Concurrency, and Hermeticity

An optional `?sid=` scopes every route to a session, so many humans can share one deployment without colliding, while the harness keeps using the implicit `default` session and its one-episode-per-task guarantee. You scale throughput by running more hermetic containers, not by adding shared state.

Learning objectives. You will understand the session model and why it was added, how it preserves the harness’s original semantics, and why hermeticity makes container-level parallelism the right scaling story.

8.1 Why sessions exist

The original design was single-episode-per-task: one global slot per task, resettable by the harness. That is perfect for evaluation, where one container runs one agent through one episode at a time. It is wrong for a *public demo*, where several people might play the same task simultaneously and would otherwise stomp on one another’s state through the single shared slot.

The resolution is a session id threaded through every task and API route as an optional query parameter:

```
def _sid(request: Request) -> str:
    sid = request.query_params.get("sid", "default")
    return sid[:64] if sid else "default"
```

Episodes are keyed by `(sid, task_id)`. Absent a `sid`, everything lives in the `default` session — exactly the original single-slot behavior the harness relies on. The interactive `/play` page mints a random session per visitor, so concurrent humans are isolated, while the harness, passing no `sid`, is untouched by the feature’s existence.

DESIGN PRINCIPLE.

Add multi-tenancy as an *optional* scoping that defaults to the prior single-tenant behavior. The new capability (concurrent isolated sessions) and the old guarantee (one deterministic episode per task for the harness) then coexist without the harness code changing at

all — backward compatibility as a default argument.

8.2 Isolation, tested

Session isolation is a correctness property, so it has a test. Two visitors reset the same task at the same seed under different session ids; one archives an email; the other's state, and the harness's `default` state, must be untouched:

```
def test_sessions_are_isolated():
    client.post("/api/email-triage/reset?sid=alice", json={"seed": 1})
    client.post("/api/email-triage/reset?sid=bob", json={"seed": 1})
    alice_state = client.get("/api/email-triage/state?sid=alice").json()["state"]
    target = next(e for e in alice_state["emails"]
                  if e["sender"] == alice_state["target_sender"])
    client.post("/api/email-triage/action?sid=alice",
               json={"op": "archive", "id": target["id"]})
    bob_state = client.get("/api/email-triage/state?sid=bob").json()["state"]
    assert not any(e["archived"] for e in bob_state["emails"])
```

Because Alice and Bob reset the *same* seed, this also incidentally re-confirms determinism: their initial inboxes are identical, and only Alice's diverges after she acts.

8.3 Scaling: more containers, not more sharing

The session model handles concurrent *humans* on a demo. It is emphatically not how you scale *evaluation*. For batch evals the guidance is blunt: one episode per task per container; parallelize by running multiple containers. This falls directly out of hermeticity (Principle 5). A container with no external dependency and no shared mutable state is trivially replicable — spin up N of them and you have N independent shards with nothing to contend over, no database to overload, no rate limit to trip. The scaling axis is horizontal and embarrassingly parallel by construction.

Contrast this with a benchmark whose tasks hit a shared live service. There, parallelism is bounded by that service — its rate limits, its statefulness, its willingness to be hammered. Hermeticity converts a coordination problem into a provisioning problem, which is the easier of the two.

Exercises

8.1 The `default` session is what makes the session feature invisible to the harness. Walk through what would break if `_sid` returned a fresh random id when `sid` was absent

instead of the constant `"default"`.

- 8.2 Session ids are truncated to 64 characters. What class of problem is this guarding against on a public deployment, and how does the LRU cap of Chapter 3 complement it?
- 8.3 Explain why “run more containers” is a satisfying scaling story here but would not be for a benchmark built on a shared, stateful third-party website.

PART III

The Harness

The Agent Interface and Action Vocabulary

An agent is anything with `begin` and `step`: it receives an `Observation` (instruction, step index, screenshot, feedback) and returns an action dictionary shaped exactly like Anthropic's computer-use action space, plus one meta-action, `done`. The vocabulary is the contract between every agent and the executor.

Learning objectives. You will be able to describe the agent protocol and the observation it consumes, enumerate the action vocabulary, and explain why mirroring an existing action space was the right interoperability choice.

9.1 The agent protocol

Symmetric to the task protocol, an agent is defined structurally:

```
@dataclass
class Observation:
    instruction: str
    step: int
    screenshot_png: bytes
    feedback: str | None = None # error/info from executing the previous action

class Agent(Protocol):
    def begin(self, instruction: str) -> None:
        """Start a fresh episode."""
    def step(self, obs: Observation) -> dict:
        """Return the next action dict (see harness/actions.py)."""
```

`begin` resets the agent for a new episode; `step` maps an observation to an action. The observation is deliberately spare and deliberately honest about what a real computer-use model gets: the instruction, which step this is, the current screenshot as PNG bytes, and any `feedback` string produced by executing the previous action (a validation error, or “that action is not supported here”). No DOM, no accessibility tree, no privileged hints — a model agent sees pixels and words, nothing more.

DESIGN PRINCIPLE.

The observation defines the agent’s epistemic situation, and it must match what you are actually trying to measure. WebTask Arena measures screenshot-driven agents, so the observation is a screenshot. Smuggling structured page data into the observation would measure a different, easier problem while pretending to measure this one.

9.2 The action vocabulary

Actions are plain dictionaries in the same shape as Anthropic’s computer-use tool output — `{"action": "left_click", "coordinate": [x, y]}`, `{"action": "type", "text": ...}`, and so on. The executor implements a fixed set, and the harness adds exactly one meta-action:

```
EXECUTABLE = {
    "screenshot", "left_click", "right_click", "middle_click",
    "double_click", "triple_click", "mouse_move", "left_click_drag",
    "type", "key", "scroll", "wait",
}
META = {"done"} # the agent believes the task is complete; verify decides
```

Two things deserve emphasis. First, `done` is the agent’s *claim* of completion, not a fact; `verify` adjudicates it. An agent that says `done` too early is not stopped from doing so — its premature termination simply becomes a gradable failure (a category in Chapter 17). Second, `screenshot` is accepted but is a no-op, because the runner screenshots before every step regardless; an agent asking to “look” costs it a turn but changes nothing, which faithfully mirrors the real tool.

9.3 Validation before execution

Between the agent’s output and the browser sits a validator that rejects malformed actions with a human-readable reason, which is fed back to the agent as next-step `feedback`:

```
def validate(action: dict) -> str | None:
    name = action.get("action")
    if name in META:
        return None
    if name not in EXECUTABLE:
        return f"unsupported action '{name}'"
    if name in {"left_click", ..., "scroll"} and "coordinate" not in action:
        return f"action '{name}' requires 'coordinate': [x, y]"
    if name in {"type", "key"} and not action.get("text"):
        return f"action '{name}' requires 'text'"
    ...
    return None
```

Validation returns `None` for a well-formed action and an error string otherwise. The runner

(Chapter 11) turns that string into the next observation's *feedback*, closing a corrective loop: an agent that emits a *left_click* without coordinates is *told* so and can recover, exactly as it would against a real tool that rejected the call.

DESIGN PRINCIPLE.

Mirror an existing, real action space rather than inventing your own. Because the vocabulary is Anthropic's computer-use shape, the Claude adapter passes model output through *unchanged* (Chapter 13), and adapters for other model families (UI-TARS, Qwen-VL) need only translate *into* this shape. The harness becomes a lingua franca instead of a private dialect.

Exercises

- 9.1 *done* is a claim, not a command to stop being graded. Explain how this design lets the harness measure "premature termination" as a distinct failure category rather than silently accepting the agent's judgment.
- 9.2 The *screenshot* action is a no-op because the runner always screenshots first. What is gained by nonetheless *accepting* it rather than rejecting it as unsupported?
- 9.3 Feedback from a rejected action becomes the next observation's *feedback* field. Describe the corrective loop this creates for an agent that repeatedly emits a *type* action with empty text.

The Browser Executor

A thin Playwright wrapper turns action dictionaries into real mouse and keyboard events at a fixed 1280×800 viewport. It translates xdotool-style key names to Playwright's, waits out load latency on navigation, and exposes a DOM oracle (`center`) reserved strictly for the reference agent — never a model.

Learning objectives. You will understand how each action becomes a browser event, why the fixed viewport matters, how key translation bridges two naming conventions, and the strict rule governing the DOM oracle.

10.1 Pixels in, events out

The `Browser` class launches Chromium at a fixed viewport and drives a single page. Its two agent-facing methods are `screenshot` (returns PNG bytes) and `execute` (applies an action). `execute` is a dispatch over the action vocabulary; the click family shows its texture:

```
if name in {"left_click", "right_click", "middle_click",
            "double_click", "triple_click"}:
    x, y = action["coordinate"]
    button = {"right_click": "right", "middle_click": "middle"}.get(name, "left")
    clicks = {"double_click": 2, "triple_click": 3}.get(name, 1)
    modifiers = [_translate_key(m) for m in action.get("text", "").split("+") if m]
    for m in modifiers:
        page.keyboard.down(m)
    page.mouse.click(x, y, button=button, click_count=clicks)
    for m in reversed(modifiers):
        page.keyboard.up(m)
    return None
```

A single branch handles five click variants by deriving the button and the click count from the action name, and supports modifier-held clicks (e.g. control-click) by pressing modifiers down, clicking, and releasing them in reverse. `type` keys text with a small inter-key delay; `scroll` converts a direction and amount into a wheel event; `wait` sleeps, capped so an agent cannot stall an episode indefinitely. An unrecognized action returns a feedback string rather than raising, so a stray action degrades to a message the agent can learn from.

DESIGN PRINCIPLE.

The executor is intentionally dumb. It contains no task knowledge and no heuristics — it is a faithful, mechanical translation from the action vocabulary to browser events. All intelligence belongs to the agent; all ground truth belongs to the server; the executor is merely the hands.

10.2 The fixed viewport

The viewport is a module constant, `VIEWPORT = (1280, 800)`, and the same constant is handed to the Claude adapter as the tool's display dimensions. This matters more than it looks. A computer-use model emits *pixel coordinates*; those coordinates are only meaningful relative to a known display size. Fixing the viewport — and telling the model the exact size it is acting on — is what makes a coordinate like `[961, 240]` well-defined. A variable viewport would make the action space itself ambiguous.

10.3 Bridging two key vocabularies

Computer-use models, following the `xdotool` lineage, emit key names like `return`, `page_down`, `ctrl`. Playwright wants `Enter`, `PageDown`, `Control`. A translation table bridges them, and a small parser handles combinations:

```
_KEY_MAP = {"return": "Enter", "enter": "Enter", "tab": "Tab",
            "escape": "Escape", "page_down": "PageDown", "ctrl": "Control",
            "super": "Meta", "cmd": "Meta", ...}

def _translate_key(combo: str) -> str:
    parts = [p.strip() for p in combo.replace(" ", "+").split("+") if p.strip()]
    out = []
    for p in parts:
        mapped = _KEY_MAP.get(p.lower())
        out.append(mapped if mapped else p)
    return "+".join(out)
```

This is unglamorous glue, but it is exactly the kind of impedance-matching that determines whether a real model's outputs land as intended. A model that presses `return` to submit a form must have that become `Enter`, or the form never submits and the failure looks like a reasoning error when it is really a translation bug.

10.4 The DOM oracle, and its one rule

`Browser` exposes two methods that consult the DOM directly:

```
def center(self, selector, has_text=None) -> tuple[float, float]:
```

```
"""Center of an element's bounding box. Scripted reference agent only."""
...
def exists(self, selector, has_text=None) -> bool:
...
```

`center` returns the pixel center of an element located by CSS selector — precisely the information a model is *not* allowed to have, since its whole challenge is to find that location from pixels. The class docstring states the rule without hedging: these are for the scripted reference agent only and must never be exposed to a model under evaluation. The enforcement is structural — the agent factory withholds the browser handle from the Claude adapter (Chapter 13) — but the discipline starts here, at the definition.

The single most damaging mistake you could make in this codebase is to let a model agent reach `center`. It would convert a hard perception-and-grounding problem into a trivial DOM lookup, and every pass rate you reported would be a fiction. The oracle exists to validate the *pipeline*, never to help an agent under test.

Exercises

- 10.1 The click branch presses modifiers, clicks, then releases them in reverse order. Why reverse order, and what could go wrong with two modifiers released in forward order?
- 10.2 `wait` is capped at a few seconds. Relate this cap to the `done` meta-action and the runner's step budget: what class of pathological agent behavior do these three mechanisms jointly bound?
- 10.3 The executor returns a feedback string for unknown actions instead of raising an exception. Argue why “degrade to feedback” is the right choice for an evaluation harness driving a fallible model.

The Episode Runner and Metrics

The runner is the conductor: reset via the oracle API, then loop screenshot → agent → validate → execute until `done` or the step budget, recording every step, then verify. Each episode writes `step_NNN.png`, `actions.jsonl`, and `result.json`; each run writes a `summary.json` with pass rates.

Learning objectives. You will be able to trace the runner’s main loop, explain the separation between the oracle channel and the agent channel, and describe the artifacts a run leaves behind and the metrics computed over them.

11.1 The main loop

One function, `run_episode`, expresses the entire lifecycle. It is worth reading nearly whole, because every earlier chapter converges here:

```
def run_episode(browser, agent, base_url, task_id, seed, opts, out_dir, max_steps):
    api = httpx.Client(base_url=base_url, timeout=15)
    reset = api.post(f"/api/{task_id}/reset",
                    json={"seed": seed, **opts}).raise_for_status().json()
    instruction = reset["instruction"]
    browser.open_task(task_id)
    agent.begin(instruction)

    feedback = None
    with open(out_dir / "actions.jsonl", "w") as log:
        for step in range(max_steps):
            png = browser.screenshot()
            (out_dir / f"step_{step:03d}.png").write_bytes(png)
            action = agent.step(Observation(
                instruction=instruction, step=step,
                screenshot_png=png, feedback=feedback))
            log.write(json.dumps({"step": step, "action": action,
                                "feedback_in": feedback}) + "\n")
            if action.get("action") == "done":
                break
        feedback = action_spec.validate(action)
```

```

if feedback is None:
    feedback = browser.execute(action)
    time.sleep(SETTLE_SECONDS)

verdict = api.get(f"/api/{task_id}/verify").raise_for_status().json()
result = {"task": task_id, "seed": seed, "opts": opts,
         "instruction": instruction, "steps": steps_taken,
         "success": verdict["success"], "subgoals": verdict["subgoals"]}
(out_dir / "result.json").write_text(json.dumps(result, indent=2))
return result

```

Read it as a sentence. Reset the episode over the oracle API and capture the instruction. Open the task page in the browser and tell the agent to begin. Then, each step: screenshot, persist the screenshot, ask the agent for an action, log it, and — unless the agent said *done* — validate and execute it, feeding any resulting message forward as the next step’s *feedback*. When the loop ends, grade over the oracle API and write the result.

11.2 Two channels, never crossed

The runner talks to the environment over *two* channels, and their separation is the operational form of Principle 2. The *httpx* client speaks the *oracle* channel — *reset* and *verify* — which the agent never touches. The *browser* speaks the *agent* channel — screenshots out, actions in. The agent object is handed only *Observations* built from screenshots; it is never handed the *api* client or the browser’s DOM oracle.

DESIGN PRINCIPLE.

Keep the grading channel and the acting channel in different objects held by different parties. The runner holds both and bridges them; the agent holds neither directly. This makes “the agent cannot grade itself or peek at ground truth” a consequence of object ownership rather than of good intentions.

Between steps the runner sleeps `SETTLE_SECONDS` (0.4s) to let the page react before the next screenshot. This is the runner honestly waiting for animations and reflows to settle — the same courtesy the browser extends to load latency on navigation. It trades wall-clock time for observation fidelity, which for an evaluation harness is the right trade.

11.3 Artifacts: a complete, replayable record

Every episode produces three artifacts in its own directory:

- `step_NNN.png` — the exact screenshot the agent saw at each step;

- `actions.jsonl` — one line per step: the action emitted and the feedback that preceded it;
- `result.json` — the final verdict with the full subgoal breakdown.

Together these are a complete trajectory: what the agent saw, what it did, and how it was graded, step by step. Because the environment is deterministic, an episode identified by `(task, seed, opts)` can be re-run to reproduce a failure exactly — and the recorded frames and actions can be packaged into the interactive replay player of Chapter 18. The failure taxonomy of Chapter 17 is, operationally, the discipline of reading these artifacts.

11.4 Metrics: from booleans to a run summary

At the run level, the CLI aggregates episode results into a summary:

```
by_task = {}
for r in results:
    by_task.setdefault(r["task"], []).append(r["success"])
summary = {
    "run_id": run_id, "agent": args.agent, "opts": opts,
    "episodes": len(results),
    "pass_rate": sum(r["success"] for r in results) / max(len(results), 1),
    "per_task": {t: {"pass": sum(v), "n": len(v)} for t, v in by_task.items()},
}
```

This is deliberately simple, and simplicity here is a virtue: `pass_rate` is just successes over episodes, reported overall and per task. The `-seeds 0-4` syntax expands to a list, so running five seeds per task and reading the per-task fractions is the natural idiom. The roadmap's `pass@1` / `passk` reporting builds on exactly this substrate — repeating seeds and aggregating the booleans — which the next section defines precisely.

11.5 From pass rate to `passk`

Two metrics recur in agent evaluation, and they answer different questions. *Pass@k* asks: across *k* attempts at a task, did *at least one* succeed? It rewards a policy that occasionally gets there. *Pass^k* asks: across *k* attempts, did *all* succeed? It rewards reliability and punishes variance. For per-attempt success probability *p* on a task,

$$\text{pass}@k = 1 - (1 - p)^k, \quad \text{pass}^k = p^k.$$

The two diverge sharply as *k* grows: `pass@k` rises toward 1 (any luck eventually pays off), while `passk` falls toward 0 (any flakiness eventually bites). An agent you would deploy is one whose `passk` stays high — consistency, not occasional brilliance, is what an evaluation aimed at trust should reward. Because the runner already records a boolean per `(task, seed)`, estimating both from repeated seeds is pure aggregation over the recorded results.

Report pass^k , not just $\text{pass}@1$, when the question is “can I trust this agent to do the task every time?” A single-attempt pass rate hides exactly the variance that a computer-use deployment cannot tolerate.

Exercises

- 11.1 The runner logs `feedback_in` alongside each action. Why is recording the feedback the agent *received* (not just the action it emitted) essential to later diagnosing a failure from `actions.jsonl` alone?
- 11.2 An agent scores $p = 0.8$ per attempt on a task. Compute $\text{pass}@3$ and pass^3 . Which would you cite to a team deciding whether to ship, and why?
- 11.3 The runner sleeps 0.4s between steps. Describe a task or perturbation for which this is too short, and how you would detect that it is too short from the recorded artifacts.
- 11.4 Trace how a single malformed action from the agent flows through `validate`, becomes `feedback`, and reappears in both `actions.jsonl` and the next `Observation`.

PART IV

Agents

The Scripted Oracle Agent

The scripted agent parses the instruction and locates targets through the DOM oracle, emitting ordinary click/type/key actions. It exists to validate the pipeline end to end — env, screenshots, executor, server, verify — not to be a fair baseline. It passes all three tasks across seeds, clean and adversarial.

Learning objectives. You will understand the purpose of an oracle agent, how it uses privileged DOM access legitimately, and why its success validates the environment rather than any model.

12.1 Why an oracle agent exists

Before you can trust a pass rate for a *model*, you must know the pipeline itself works: that reset produces a solvable episode, that screenshots flow, that actions execute, that server state updates, and that verify grades correctly. The scripted agent is that proof. It is allowed the one thing a model is forbidden — DOM access via `browser.center` — precisely because its job is to exercise the plumbing, not to demonstrate perception.

DESIGN PRINCIPLE.

Build a privileged reference solver whose success certifies the *environment* and whose privilege is exactly the capability the models under test lack. When the oracle passes and a model fails, the difference localizes cleanly to what the oracle had and the model did not — perception and grounding — because everything else in the pipeline is proven to work.

12.2 Plans as generators

Each task gets a plan: a Python generator that yields actions one at a time. `begin` selects the plan by task id; `step` simply advances it, and returns `done` when the generator is exhausted:

```
def step(self, obs: Observation) -> dict:
    try:
        return next(self._plan)
    except StopIteration:
```

```
return {"action": "done"}
```

The email plan reads its targets from the instruction (never the oracle API), then archives while any target rows remain and stars the subject-matched row:

```
def _email_plan(self, instruction):
    m = re.search(r"archive every email from (.+?) and star the email "
                  r"with the subject . (.?) .", instruction)
    sender, subject = m.group(1), m.group(2)
    while self.browser.exists("tr.email-row", has_text=sender):
        row = f"tr.email-row:has(td.sender:text-is('{sender}'))"
        x, y = self.browser.center(f"{row} .archive-btn")
        yield {"action": "left_click", "coordinate": [round(x), round(y)]}
        yield {"action": "wait", "duration": 0.3}
    star_row = f"tr.email-row:has-text('{subject}')"
    yield self._click(f"{star_row} .star-btn")
```

Two details reward attention. First, the plan parses the *instruction*, exactly as a model would have to — its only shortcut is where it clicks, not what it knows. Second, the `while ... exists` loop re-queries the DOM on every iteration, so it naturally follows the list as it reflows under archiving. That is why the oracle survives the very reflow that breaks a coordinate-fixed model in Chapter 16: it re-grounds every action, which is precisely the capability the environment is designed to test.

12.3 The settings plan and a subtle lesson

The settings plan navigates to the Appearance section, toggles Dark Mode, returns to General, and drives the timezone `<select>` by arrowing down from UTC:

```
yield self._click("#nav button", has_text="Appearance")
yield self._click("input[data-field='dark_mode']")
yield self._click("#nav button", has_text="General")
yield self._click("select[data-field='timezone']")
for _ in range(TIMEZONES.index(target_tz)):
    yield {"action": "key", "text": "Down"}
yield {"action": "key", "text": "Return"}
yield self._click("#save")
```

The oracle solves the timezone by counting arrow-key presses from a known index — a strategy available to it because it *knows* the option order (a documented constant mirroring the server's). A screenshot model has no such luxury and, as the case study shows, cannot even see the option list under headless rendering. The oracle's success on this task therefore certifies that the task is solvable and correctly graded, while foreshadowing exactly where a real agent will struggle.

The oracle passes all three tasks across seeds under both clean and adversarial rendering. Read that result carefully: it says the *environment* is sound — solvable, deterministic, correctly graded, and robust in the sense that a re-grounding solver is not defeated by the perturbations. It says *nothing* about any model. Never quote the oracle’s pass rate as a baseline; it is a self-test of the arena.

Exercises

- 12.1 The scripted agent is allowed `browser.center` but a model is not. State, in one precise sentence, what the oracle’s passing does and does not certify.
- 12.2 The email plan’s `while ... exists` loop is the reason it survives list reflow. Rewrite the loop in the naive coordinate-caching style that would fail, and explain the exact step at which it goes wrong.
- 12.3 The settings plan hardcodes the timezone option order as a constant that must match the server. Is this duplication acceptable? Argue for a design that would remove it, and a reason the current design keeps it anyway.

The Claude Computer-Use Adapter

The Claude adapter runs the standard computer-use loop: it sends the instruction and screenshots, receives `computer_20251124` tool calls, and passes their input through to the executor unchanged. It holds no browser handle — the factory withholds it — so the model can never reach the DOM oracle.

Learning objectives. You will understand how a real computer-use model is wired into the harness, how tool-use results and screenshots are threaded through the conversation, and how the design enforces the agent/oracle boundary for a live model.

13.1 Zero-translation by design

Because the harness action vocabulary *is* Anthropic’s computer-use shape (Chapter 9), the adapter performs no translation on the way out. It extracts the model’s tool-use block and returns its `input` dict directly to the runner, which executes it:

```
tool_use = next((b for b in response.content if b.type == "tool_use"), None)
if tool_use is None:
    self._pending_tool_use = None
    return {"action": "done"}
self._pending_tool_use = tool_use.id
return dict(tool_use.input)
```

When the model stops calling the tool — it has decided the task is done — the adapter returns the `done` meta-action. This is the payoff of mirroring the real action space: the “adapter” is almost a pass-through, and the interesting behavior is the model’s, not the glue’s.

13.2 Threading screenshots and tool results

A computer-use conversation alternates: the model emits a tool call, the environment returns a tool *result* containing the new screenshot, and the model emits the next call. The adapter maintains this message history. On the first step it attaches the initial screenshot to the instruction turn; on subsequent steps it wraps the previous action’s feedback and the new screenshot in a `tool_result` keyed to the pending tool-use id:

```

if self._pending_tool_use is None:
    self.messages[-1]["content"].append(image_block) # first screenshot
else:
    content = []
    if obs.feedback:
        content.append({"type": "text", "text": obs.feedback})
    content.append(image_block)
    self.messages.append({"role": "user", "content": [{
        "type": "tool_result",
        "tool_use_id": self._pending_tool_use,
        "content": content}]}))

```

The request itself declares the computer tool at the harness's exact viewport, so the model's pixel coordinates and the executor's viewport agree:

```

response = self.client.beta.messages.create(
    model=self.model, max_tokens=self.max_tokens, betas=[BETA],
    thinking={"type": "adaptive"}, system=SYSTEM,
    tools=[{"type": "computer_20251124", "name": "computer",
        "display_width_px": self.viewport[0],
        "display_height_px": self.viewport[1]}],
    messages=self.messages)
self.messages.append({"role": "assistant", "content": response.content})

```

The adapter appends the assistant's *full* content — including thinking blocks — back into the history. This is required for a faithful multi-turn computer-use loop and it is also what makes a later replay show the model's reasoning next to each action. Preserving the model's own record is both a correctness requirement and an interpretability gift.

13.3 The boundary, enforced by construction

The most important line about the Claude adapter is one it does *not* contain: it never receives the browser. Recall the factory:

```

def make_agent(name: str, **kwargs):
    if name == "scripted":
        return ScriptedAgent(**kwargs) # gets the browser
    if name == "claude":
        from .claude import ClaudeComputerUseAgent
        return ClaudeComputerUseAgent(
            **{k: v for k, v in kwargs.items() if k != "browser"})

```

The scripted agent is constructed *with* the browser (it needs the DOM oracle); the Claude adapter is constructed with the browser argument explicitly filtered out. The model therefore

has no object through which it could reach `center` or `exists`. Principle 2 and the DOM-oracle rule are thus enforced not by convention but by what the model is physically handed.

DESIGN PRINCIPLE.

Enforce a security boundary by controlling capability distribution, not by trusting call sites. The model cannot misuse the DOM oracle because it never holds a reference to it. A boundary you can violate only by rewriting the factory is far stronger than one you must remember to respect.

At the time of first writing, the adapter was built but not yet exercised against the live API — the first real datapoint awaited an API key. Being candid about that status is itself good evaluation practice: an untested adapter is a plausible result generator, not a result. The next chapter is a case study of a model that *was* run end to end.

Exercises

- 13.1 The adapter returns `done` exactly when the model emits no tool-use block. Relate this to the model's system prompt instruction to "stop calling tools when finished," and to how `verify` then adjudicates the claim.
- 13.2 Explain why declaring `display_width_px/display_height_px` equal to the executor's viewport is not optional. What failure mode appears if they disagree by even a small margin?
- 13.3 The factory filters `browser` out of the Claude adapter's kwargs. Show how a well-meaning refactor could accidentally restore the browser to the model, and propose a test that would catch it.

Porting New Agents

A new agent family joins the arena by translating its native output *into* the harness action vocabulary and implementing `begin/step`. Because the vocabulary mirrors a real action space, translation is the whole job; the executor, runner, tasks, and grader are untouched.

Learning objectives. You will be able to outline the work required to add an agent family such as UI-TARS or Qwen2.5-VL, and identify the pitfalls that translation introduces.

14.1 The shape of the work

Adding an agent is symmetric to adding a task: implement a narrow protocol and register it in the factory. Concretely:

1. Implement `begin(instruction)` and `step(obs) -> dict`, returning actions in the harness vocabulary.
2. Inside `step`, run the model on `obs.screenshot_png` and `obs.instruction`, then *translate* the model's native action into a `{"action": ...}` dict — mapping its click/type/scroll primitives onto the executor's names and coordinate convention.
3. Add a branch to `make_agent`. Withhold the browser handle unless the agent is a privileged oracle.

For models whose native output already resembles computer-use (as Claude's does), translation is near-trivial. For a model like UI-TARS, whose transcript encodes actions as lines such as `click point=(x, y)`, translation means parsing that transcript into the dict form — which is exactly what the replay tooling of the companion evaluation does.

DESIGN PRINCIPLE.

When you mirror a real action space at the boundary, onboarding a new model collapses to a translation function. The cost of supporting the n -th agent family is $O(\text{that family's output format})$, not $O(\text{the whole harness})$. This is the interoperability dividend of Principle in Chapter 9.

14.2 Translation pitfalls

Translation is simple but not free, and its pitfalls are precisely the ones that masquerade as model errors.

Coordinate conventions. If a model emits coordinates in a normalized $[0, 1]$ space, or relative to a different display size than the executor's viewport, every click lands in the wrong place and the failure looks like poor grounding. Always reconcile the model's coordinate space to the executor's fixed 1280×800 before emitting the action.

Key naming. A model that emits `ENTER` or `Return` or `\\n` must have it land as the executor's `key` with a name the translator recognizes, or form submissions silently fail. The key map of Chapter 10 is the reconciliation point; extend it rather than guessing.

Termination. Different models signal “done” differently — a special token, an empty action, a natural-language sentence. Map each family's completion signal onto the `done` meta-action deliberately; a missing mapping turns every episode into a step-budget timeout.

The lesson of Chapter 16 is that these translation and rendering issues are not footnotes: a real evaluation of UI-TARS surfaced an *environment* finding (invisible `<select>` options) and a *model* finding (state-tracking failure) side by side, and telling them apart is the core skill of running an arena.

Exercises

- 14.1 A new model reports coordinates as fractions of the screen in $[0, 1]$. Write the two-line translation that reconciles them to the executor's viewport, and state what goes wrong if you forget it.
- 14.2 Which of the three translation pitfalls would most plausibly be misdiagnosed as a reasoning failure by someone reading only the pass rate? Justify.
- 14.3 Sketch the `step` method of an adapter for a model whose native output is a single line of text like `type text="hello"` or `click (410, 220)`. Include how you detect and emit `done`.

PART V

Evaluation and Findings

The Contract Test Suite

The tests are not incidental quality assurance; they are the executable specification of the five principles. They assert determinism, the un-solved initial state, session isolation, and — crucially — that each task’s designed trap actually fails an agent that falls into it.

Learning objectives. You will be able to categorize the suite’s tests by the property each defends, and explain why testing the *traps* is as important as testing the happy path.

15.1 Determinism and sanity

The suite opens with the properties the whole system rests on. We have already seen `test_same_seed_same_episode` (Chapter 4). Its companions assert that seeds actually vary the episode, that a freshly reset episode is *not* already solved, and that every task page renders:

```
@pytest.mark.parametrize("task_id", ALL_TASKS)
def test_fresh_episode_is_not_solved(task_id):
    reset(task_id, seed=3)
    assert verify(task_id)["success"] is False
```

`test_fresh_episode_is_not_solved` is more important than it looks. A task whose initial state already satisfied its own reward would report spurious successes for any agent, including one that did nothing. Asserting non-triviality of the start state is asserting that the task measures *something*.

DESIGN PRINCIPLE.

Test that success is *reachable but not free*: the fresh episode must fail verification, and a correct sequence of actions must pass it. Together these bracket the reward — it is neither vacuously true nor impossible — which is the minimum condition for a task to be a measurement at all.

15.2 Testing the traps

The most distinctive tests are the ones that deliberately fall into each task’s trap and assert that the reward catches it. These encode the task’s *reason for existing*. Email precision:

```
def test_email_triage_extra_archive_fails():
    reset("email-triage", seed=42); solve_email()
    extra = next(e for e in oracle("email-triage")["emails"]
                 if e["sender"] != state["target_sender"] and not e["archived"])
    act("email-triage", {"op": "archive", "id": extra["id"]})
    result = verify("email-triage")
    assert result["success"] is False
    assert result["subgoals"]["no_extra_archived"] is False
```

Settings commitment — toggling without saving must not count — and settings non-interference — an unrelated change must fail — each have a test. Checkout has the sharpest of all, asserting the UI-lies property directly:

```
def test_checkout_wrong_data_submits_but_fails_verify():
    reset("checkout-form", seed=5)
    act("checkout-form", {"op": "contact", "name": "Wrong Person", "email": "wrong@example.com"})
    act("checkout-form", {"op": "shipping", "street": "1 Nowhere St", "city": "Nil", "zip": "00000"})
    act("checkout-form", {"op": "submit"})
    result = verify("checkout-form")
    assert result["subgoals"]["submitted"] is True
    assert result["success"] is False
```

`submitted is True` while `success is False` is Principle 2 rendered as an assertion: the order went through the UI, and the server still knows it is wrong.

DESIGN PRINCIPLE.

For every trap a task is designed to set, write a test that springs it. A reward function is only as trustworthy as your evidence that it fails the failures. Testing only the happy path certifies that a correct agent passes; testing the traps certifies that an incorrect one does not — and the second is the harder, more valuable guarantee.

15.3 Validation and ordering

Checkout’s server-side validation and step ordering are tested too: bad input is rejected with field-level errors, and steps cannot be skipped — `test_checkout_cannot_skip_steps` asserts a bare `submit` returns HTTP 400. These confirm that `handle_action` enforces the interface’s rules, so that a model’s recovery from a validation error is a real capability the task exercises rather than a formality the server waves through.

Because the determinism and sanity tests are *parametrized over the registry*, a newly added task inherits them for free (Chapter 5). The trap tests, by contrast, are necessarily task-specific — each trap is unique — and must be written by hand for each new task. The division tells you which guarantees are structural and which are earned.

Exercises

- 15.1 Classify each of the following as defending determinism, non-triviality, isolation, or a trap: `test_different_seeds_differ`, `test_settings_unsaved_draft_does_not_count`, `test_sessions_are_isolated`, `test_fresh_episode_is_not_solved`.
- 15.2 Write the trap test you would add for the “compose and send an email” task you designed in Chapter 5’s exercises. What incorrect agent behavior does it spring?
- 15.3 The suite tests both that a fresh episode fails verify and that a solved one passes. Explain, with reference to a hypothetical buggy reward, why either test alone would be insufficient.

Case Study: UI-TARS-1.5-7B in the Arena

Running a real 4-bit UI-TARS model through the arena at seed 0 produced one pass of three, and — more valuably — two precisely diagnosed failures: a model-side state-tracking collapse on email-triage, and an environment-side rendering finding on settings-panel. Distinguishing the two is the entire point.

Learning objectives. You will study a real evaluation result, practice separating model failures from environment findings, and see how the recorded artifacts support a specific diagnosis rather than a vague one.

16.1 The result, in one line each

A quantized UI-TARS-1.5-7B was driven through all three tasks at seed 0. The per-task diagnoses, recorded in the replay manifest, read:

- **checkout-form — pass.** “Clean run: precise field clicks, correct typing, three wizard steps, submit.” The model handled multi-step form filling end to end.
- **email-triage — fail (model).** “State-tracking failure: archives worked, but the model never perceived rows disappearing — it re-clicked fixed coordinates while the list reflowed underneath, archiving 7 extra emails while blaming ‘an unresponsive system.’”
- **settings-panel — fail (environment).** “Environment finding: headless Chromium renders native `<select>` dropdowns outside the page — the timezone options are invisible to any screenshot agent. Dark Mode itself was correctly enabled and saved.”

16.2 Anatomy of a model failure

The email-triage subgoals tell the story precisely: `archived_all_from_target true`, `no_extra_archived false`, `starred_target_only false`. The model achieved recall and destroyed precision — it archived the targets and then kept clicking the same coordinates as the list reflowed beneath it, archiving seven innocent emails. This is exactly the failure the email task

was built to catch (Chapter 6) and exactly the capability the scripted oracle’s re-querying loop was designed to have and this model lacked (Chapter 12).

The model narrated its own failure as “an unresponsive system” — it perceived its clicks as ineffective when in fact they were landing on the wrong, reflowed rows. This is a textbook *state-tracking* failure: the agent’s internal model of the screen desynchronized from the actual screen, and it never took a fresh observation to re-ground. The precision subgoal converted this narrated confusion into a hard, gradable fact.

16.3 Anatomy of an environment finding

Settings-panel is different in kind. The subgoals: `dark_mode_saved` true, `no_unrelated_changes` true, `timezone_saved` false. The model did everything it could see to do — it enabled and saved Dark Mode, disturbed nothing else — and failed only the timezone. The diagnosis is not “the model is bad at dropdowns.” It is that headless Chromium renders a native `<select>` option list *outside the page canvas the screenshot captures*, so the options were never in any image the model received. No screenshot agent could have succeeded.

DESIGN PRINCIPLE.

Not every failed subgoal is the agent’s fault. An *environment finding* is a case where the harness, not the policy, made the task unwinnable — here, a rendering interaction between a native control and headless mode. An honest arena reports these as findings about *itself* and fixes or brackets them; attributing them to the model would be a measurement error dressed as a result.

This finding is itself valuable output. It says: if you want to test dropdown interaction with screenshot agents, do not use a native `<select>` under headless rendering; use a custom in-page dropdown, or drive a headed browser. That is a concrete, reusable lesson for anyone building GUI-agent environments — precisely the kind of contribution controlled failure analysis is meant to yield.

16.4 Why one seed still teaches

A single seed is not a statistic, and the case study does not pretend otherwise — the roadmap’s pass^k-across-seeds work is exactly the fix. But one carefully read seed already yields three distinct, actionable conclusions: that the model can do multi-step forms, that it has a specific and diagnosable state-tracking weakness, and that the environment has a specific and fixable rendering pitfall. The lesson of the chapter is methodological: *reading* a trajectory closely — screenshots, actions, the model’s own words, and the subgoal vector together — extracts far more than a pass rate ever could.

Exercises

- 16.1 Given only the subgoal vector `{archived_all_from_target: true, no_extra_archived: false, starred_target_only: false}`, reconstruct the most likely failure narrative without reading the diagnosis. What additional artifact would confirm it?
- 16.2 You observe `timezone_saved: false` for a new model. List the checks, in order, you would run to decide whether this is the known environment finding or a genuine model failure.
- 16.3 The checkout pass and the email failure came from the *same model* at the *same seed*. What does their juxtaposition tell you about the nature of the email failure that a low pass rate alone would not?

Toward a Failure Taxonomy

The point of the arena is a taxonomy of failure. Four recurring categories — grounding misclick, perception error, planning loop, and premature termination — turn “it failed” into “it failed *this way*.” Named subgoals and recorded trajectories are the evidence base; the taxonomy is the vocabulary.

Learning objectives. You will be able to define each failure category, map it to the subgoal and trajectory evidence that identifies it, and explain why a taxonomy is more useful than a scalar score for improving an agent.

17.1 Four categories

Controlled failure analysis converges on a small set of recurring failure modes. The four the arena is built to distinguish:

Grounding misclick. The agent knows *what* to click but not *where*: it targets the right element conceptually and lands on the wrong pixels — often after a layout shift or reflow. Evidence: actions aimed at plausible but stale coordinates; success on “what” subgoals, failure on ones that require hitting the exact target.

Perception error. The agent misreads the screen: it cannot see a control (the invisible `<select>`), misidentifies state, or fails to notice a change. Evidence: actions that presuppose a screen different from the recorded screenshot.

Planning loop. The agent repeats an ineffective action, unable to make progress, sometimes narrating growing frustration. Evidence: cycles in `actions.jsonl`, repeated identical actions, exhaustion of the step budget.

Premature termination. The agent emits `done` before the task is actually complete. Evidence: `done` with unmet subgoals and steps remaining in the budget.

17.2 From subgoals to categories

The named subgoals are the taxonomy’s raw signal. Consider the case study’s email failure: recall passed, precision failed, star failed. Combined with the trajectory — repeated clicks at fixed coordinates as the list reflowed — this vector points unambiguously at a *grounding misclick* compounded by a *state-tracking* (perception) error, not a planning loop or an early stop. Contrast the settings failure: a single clean subgoal miss with everything else correct, and a trajectory showing the option list was never visible — a *perception error*, and moreover an environment-induced one.

DESIGN PRINCIPLE.

A subgoal vector plus a trajectory identifies a failure *category*; neither alone suffices. The vector says which capability broke; the trajectory says how. Designing subgoals so their failure patterns discriminate between categories is therefore part of building a good arena — the reward is not just a grade but a diagnostic instrument.

17.3 Why a taxonomy beats a score

A scalar pass rate answers “how good?” A taxonomy answers “what to fix?” If 80% of an agent’s failures are grounding misclicks after layout shifts, the intervention is about re-grounding and stability, not about reasoning. If they are premature terminations, the intervention is about completion criteria. The taxonomy routes engineering effort; the score cannot.

The failure taxonomy is the currency of evaluation work. Leaderboards rank; a taxonomy *explains*, and explanation is what lets a team improve a model rather than merely observe it. The arena’s deliverable is annotated failures, not a number.

17.4 Building the taxonomy pipeline

Operationally, the taxonomy is a pipeline over recorded artifacts. For each failed episode: read the subgoal vector to localize the broken capability; scan `actions.jsonl` for cycles, stale coordinates, or an early `done`; and inspect the `step_NNN.png` frames around the divergence. Annotate the episode with a category and a one-line diagnosis — exactly the form the case study’s manifest already uses. Aggregate categories across seeds and conditions and you have a *distribution of failure modes*, which is the arena’s headline result. The adversarial flags (Chapter 7) sharpen this: comparing the category distribution clean versus perturbed shows which failures are latent and which the perturbation induces.

Exercises

- 17.1 For each of the four categories, name the single most discriminating piece of evidence (a subgoal pattern or a trajectory feature) that distinguishes it from the other three.
- 17.2 An agent times out (hits the step budget) on a task. Which two categories are candidates, and what one artifact would let you decide between them?
- 17.3 Propose a fifth failure category not in the list, define its evidence signature, and give a task from the arena on which you would expect to observe it.
- 17.4 Explain how comparing the failure-category distribution under clean versus adversarial rendering could reveal that an agent's clean-condition success was fragile. What pattern would you look for?

Replays and the Human Baseline

Recorded episodes become an interactive replay player — every screenshot, action, and model thought beside the server’s verdict — and a “beat the agent” mode where a human plays the same seeded episode and is graded by the same oracle, subgoal by subgoal. Determinism is what makes both possible.

Learning objectives. You will understand how trajectories are packaged for replay, why a same-seed human baseline is a meaningful comparison, and how the demo surfaces reinforce the book’s principles for a non-expert audience.

18.1 Packaging a trajectory for replay

The `build_replays.py` script converts raw run directories into the player’s format: it keeps one frame per step (runners pad video with duplicates, downscales to JPEG), pairs each frame with the parsed action and the model’s thought, and attaches the subgoal verdict and a one-line diagnosis. The output is a per-task `replay.json` plus a top-level `manifest.json` that the landing and replay pages read. A single step in a replay carries everything needed to understand it:

```
{
  "img": "step_000.jpg",
  "action": "click @ (961, ...)",
  "thought": "I noticed several emails from Ava Petrov ... archive them ..."}

```

Placing the model’s own reasoning next to the action and the eventual verdict is what turns a replay from a screencast into an *explanation*. A viewer watches the state-tracking failure of Chapter 16 happen: the thoughts stay confident while the clicks drift onto reflowed rows.

18.2 The same-seed human baseline

The “beat the agent” mode lets a person play the identical seeded episode the model played, graded by the identical oracle. Because the environment is deterministic, “the same task” is a literal, defensible claim — not a similar task, the *same* inbox, the *same* target, the *same* reward. That makes the human’s subgoal-by-subgoal result a genuine baseline: it establishes that the task is humanly solvable (a sanity check on difficulty) and calibrates just how hard the parts

the model failed really are.

DESIGN PRINCIPLE.

A human baseline is only meaningful if the human faces the *same* instance the agent did. Determinism upgrades “a person can usually do this” into “a person did *this exact episode*, and here is which subgoals they hit.” The comparison is honest precisely because the seed pins the instance.

18.3 The demo as pedagogy

The public surfaces — a landing page, the replay player, and the play mode — are more than a showcase. They make the book’s abstract principles tangible to someone who will never read the source: determinism becomes “you and the model got the same inbox,” server-side reward becomes “the grader knew your order was wrong even though the page said thanks,” and the failure taxonomy becomes “watch exactly where it went off the rails.” An evaluation instrument that can explain itself to a newcomer is one whose principles are sound enough to survive translation out of code.

The session model of Chapter 8 is what makes the play mode safe: each visitor gets an isolated `sid`, so concurrent humans neither corrupt one another’s episodes nor disturb the harness’s `default` session. The demo and the evaluation harness share one codebase precisely because the session feature was added without breaking the single-episode guarantee.

Exercises

- 18.1 Why does packaging the model’s `thought` alongside each frame make the replay a better diagnostic tool than screenshots and actions alone? Tie your answer to a specific category from Chapter 17.
- 18.2 The human baseline requires the human to play the same *seed*. Describe what the comparison would and would not tell you if the human instead played a *different* seed of the same task.
- 18.3 The replay downscales frames to JPEG to save space. What evaluation question could this lossy compression compromise, and how would you preserve the information needed to answer it?

PART VI

Extensions and Research

From Evaluation to Reinforcement-Learning Environments

An evaluation environment and an RL training environment share the same skeleton: a deterministic reset, a transition function, and a reward. WebTask Arena already has all three. Turning it into an RL environment is mostly about exposing the loop as a step API and thinking hard about reward shaping and gaming.

Learning objectives. You will see how the arena’s components map onto the standard RL environment interface, understand why the “environments, rewards, evals” layer is the foundation RL builds on, and appreciate the reward-gaming risks RL amplifies.

19.1 The arena is already most of an RL environment

The reinforcement-learning abstraction is a tuple: states, actions, a transition function, and a reward. Map it onto what the arena has:

RL concept	Arena component
Initial state distribution	<code>generate(seed)</code> over seeds
Observation	the screenshot the runner captures
Action space	the computer-use action vocabulary
Transition function	<code>handle_action</code> + the browser
Reward	<code>verify</code> ’s subgoal vector
Episode reset	<code>POST /reset</code>

The pieces reinforcement learning needs are exactly the pieces a good evaluation harness already builds. This is why the “pre-RL” layer — environments, reward functions, evaluation — is the foundation the training layer stands on: you cannot train against a reward you cannot compute reliably, nor debug a policy in an environment you cannot reset deterministically.

DESIGN PRINCIPLE.

Build the environment, reward, and evaluation stack *first* and build it rigorously. Reinforcement learning inherits every weakness of the environment it trains against: a flaky reward becomes a corrupt training signal, and a nondeterministic reset becomes an unreproducible experiment. The unglamorous half is load-bearing.

19.2 What RL would add, and what it would demand

To make the arena a training environment you would expose the runner's loop as a programmatic `step(action) -> (observation, reward, done)` interface, and you would confront questions evaluation lets you defer:

Reward shaping. The subgoal vector is a sparse, end-of-episode signal — fine for grading, hard for learning. A long-horizon task with reward only at the end gives a nascent policy almost nothing to climb. The named subgoals are a natural starting point for *process* rewards (credit for reaching an intermediate subgoal), but every added shaping term is a new opportunity to be gamed.

Reward gaming. An optimizer will exploit any gap between the reward and the intent. The arena's design already anticipates this — server-side grading and restraint subgoals (no-extra-archived, no-unrelated-changes) exist precisely to make “looks done” diverge from “is done.” Under RL pressure those defenses matter far more, because a trained policy will find the cheapest path to reward, and if archiving the whole inbox scored well it would learn to.

This is the honest boundary of the current work: the arena is deliberately the *pre-RL* half of the stack — environments, rewards, evaluations — where the design is strongest and most complete. The RL training half (policy optimization, on-policy rollouts, credit assignment) is future work that this foundation is built to support, not something the current system claims to do.

19.3 Determinism in training versus evaluation

A subtlety worth flagging: evaluation wants determinism, but training wants *controlled* variation. The same seeding machinery serves both. For evaluation you fix seeds to compare policies on identical instances; for training you sample seeds to expose the policy to the task distribution, while still being able to *replay* any training episode by its seed for debugging. The seed is the control knob in both regimes — pinned for measurement, sampled for learning, and always available for reproduction.

Exercises

- 19.1 Map each element of the RL tuple onto a specific function or route in the arena, and identify the one element the current system does *not* yet expose programmatically.
- 19.2 The email task's `no_extra_archived` subgoal is a restraint term. Explain how you would incorporate it into a shaped reward, and describe a gaming strategy a trained policy might discover if you weighted it wrongly.
- 19.3 Argue why a nondeterministic reset is more damaging to RL training than to one-shot evaluation. Use the notion of an unreproducible experiment.

Research Directions

The roadmap points at more agents, richer metrics, harder interactions, and a systematic failure taxonomy across the seed $\times$ condition grid. Each direction is a small, well-scoped step that the existing architecture makes cheap — which is itself evidence the foundations were laid correctly.

Learning objectives. You will be able to situate the project’s roadmap within the principles of the book and identify which architectural decisions make each future direction inexpensive.

20.1 More agents, more coverage

Adding a Qwen2.5-VL adapter and porting the UI-TARS adapter into the harness are translation exercises (Chapter 14), cheap by design. Each new agent turns the arena into a comparative instrument: the same seeds, tasks, and reward, applied to several model families, yield a like-for-like failure-mode comparison that a shared benchmark’s noise would obscure.

20.2 Richer metrics: the seed $\times$ condition grid

The natural next measurement is $\text{pass}@1$ and pass^k with per-subgoal breakdowns across repeated seeds and both rendering conditions. The runner already emits the per-(task, seed) booleans this needs; the work is aggregation and presentation. The headline artifact is a grid — tasks and subgoals down one axis, rendering conditions across the other — whose cells are reliabilities, not single runs. That grid is where the adversarial ablation of Chapter 7 finally pays off quantitatively.

20.3 Harder interactions

The roadmap lists new tasks that stress capabilities the current three do not: drag-to-reorder (continuous control and precise release), infinite-scroll search (perception under a moving viewport), and modal interruptions (recovering attention after an unexpected dialog). Each is a new Task implementation and template — no change to the server, runner, or grader — which is the extension story of Chapter 5 working as promised.

DESIGN PRINCIPLE.

A well-designed foundation is recognizable by the cheapness of its roadmap. That every listed next step is “one adapter,” “one aggregation,” or “one task class” — never “rearchitect the harness” — is the strongest available evidence that the abstractions of Parts II and III were drawn in the right places.

20.4 The standing deliverable

Above any single feature, the project’s enduring output is the failure taxonomy of Chapter 17: grounding misclicks versus perception errors versus planning loops versus premature termination, with annotated trajectories as evidence, measured across agents, seeds, and conditions. Everything else — more agents, richer metrics, harder tasks — exists to make that taxonomy broader and better supported. The arena’s purpose is not to say which agent wins, but to say, with evidence, *how agents that operate interfaces break* — and thereby to help the next one break less.

Exercises

- 20.1 For each roadmap item (Qwen adapter, pass^k reporting, drag-to-reorder task), name the single file or module where the bulk of the work lands and the files that do *not* change.
- 20.2 Design the layout of the “seed × condition” results grid for one task. What goes in the rows, the columns, and the cells, and how would a reader spot a fragile clean-condition success?
- 20.3 Pick one proposed hard-interaction task and specify its *verify*: the state it reads, its named subgoals (including at least one restraint subgoal), and the trap it sets.

PART VII

Appendices

HTTP API Reference

All routes accept an optional `?sid=<session>` query parameter; absent it, the `default` session is used. Oracle routes must never be exposed to the agent under evaluation.

Agent surface

`GET /task/{task_id}` Render the task page. Reads the current episode (creating a default seed-0 episode if none exists). Serves the HTML the agent's browser loads.

`POST /api/{task_id}/action` Apply a UI action. Body is the task-specific action payload (e.g. `{"op":"archive","id":3}`). Returns the handler's result with HTTP 200 on `ok`, 400 otherwise. Called by the page's own JavaScript, not by the harness directly.

Oracle surface (harness only)

`POST /api/{task_id}/reset` Body: `{"seed":int, "latency_ms":int, "layout_shift":bool}`, all optional. Builds a fresh seeded episode. Returns `{"task_id","seed","instruction"}`.

`GET /api/{task_id}/instruction` Return the current episode's instruction string.

`GET /api/{task_id}/verify` Grade the episode. Returns `{"success":bool,"subgoals":{"name:bool}}` where `success` is the conjunction of subgoals.

`GET /api/{task_id}/state` Return raw privileged state: `{"seed","opts","state"}`. Debug/oracle only.

`GET /healthz` Liveness probe; returns `{"status":"ok"}`.

Demo surface

`GET /` Landing page: task links and bundled replays.

`GET /replays` Replay player (200 if replays are bundled, else 404).

`GET /play` "Beat the agent" mode; mints a random `sid` per visitor.

Action Reference

Actions are dictionaries with an `"action"` key. Executable actions map to browser events; the one meta-action is adjudicated by `verify`.

Action	Required fields	Effect
<code>left_click</code> / <code>right_click</code> <code>middle_click</code>	<code>coordinate:[x,y]</code>	mouse click; <code>text</code> may hold modifier combo (e.g. <code>"ctrl"</code>)
<code>double_click</code> / <code>triple_click</code>	<code>coordinate:[x,y]</code>	2 or 3 clicks
<code>mouse_move</code>	<code>coordinate:[x,y]</code>	move cursor
<code>left_click_drag</code>	<code>start_coordinate</code> , <code>coordinate</code>	press, move, release
<code>type</code>	<code>text</code>	type text with key delay
<code>key</code>	<code>text</code>	press a key/combo (xdotool names)
<code>scroll</code>	<code>coordinate</code> , <code>scroll_direction</code> , <code>scroll_amount</code>	wheel scroll
<code>wait</code>	<code>duration</code> (capped)	sleep
<code>screenshot</code>	—	no-op (runner always shoots)
<code>done</code>	—	claim completion; <code>verify</code> decides

Malformed actions are rejected by `validate` with a human-readable string that becomes the next observation's `feedback`.

Command Reference

```
# Run the environment
docker compose up --build          # -> http://localhost:8000

# Local dev
pip install -r requirements-dev.txt && playwright install chromium
uvicorn server.app:app --reload
pytest                            # determinism + reward contract tests

# Drive an episode by hand
curl -X POST localhost:8000/api/email-triage/reset \
  -H 'Content-Type: application/json' \
  -d '{"seed": 42, "latency_ms": 800, "layout_shift": true}'
curl localhost:8000/api/email-triage/verify

# Harness: scripted self-test (validates the pipeline)
python -m harness.run --agent scripted --task all --seeds 0-4

# Harness: same tasks under adversarial rendering
python -m harness.run --agent scripted --task all --seeds 0-4 \
  --latency-ms 800 --layout-shift

# Harness: Claude computer-use (needs ANTHROPIC_API_KEY + pip install anthropic)
python -m harness.run --agent claude --task all --seeds 0-4
```

Glossary

Adversarial rendering Optional perturbations (`latency_ms`, `layout_shift`) that stress perception and grounding without changing task logic or reward.

Environment finding A failed episode caused by the harness or rendering, not the agent — e.g. native `<select>` options invisible under headless Chromium. Reported as a fact about the arena, not the model.

Episode One attempt at one task, identified by `(task, seed, opts)`; recorded as screenshots, an action log, and a result.

Hermetic Self-contained: no external network dependency, so behavior is fully controlled by the container and the seed.

Oracle endpoint `reset`, `verify`, `state` — for the harness only; never exposed to the agent under test.

Oracle (scripted) agent The privileged reference solver that uses DOM access to validate the pipeline; not a baseline.

pass@ k / pass ^{k} At-least-one-success vs. all-successes over k attempts; the latter measures reliability.

Reward hacking Achieving reward without the intended behavior; defended against by server-side grading and restraint subgoals.

Seed Integer that fully determines an episode via `rng(task_id, seed)`.

Subgoal A named boolean component of success; the atoms of the failure taxonomy.

Failure taxonomy The structured vocabulary of how agents break: grounding misclick, perception error, planning loop, premature termination.

End of book. Read with the source open: github.com/xiningli/webtask-arena